

# The C47/R47 1000-digit issue

If you, the reader, are not into operations involving up to 1,000 digits (and some of us will happily admit that we are not), then feel free to move past this appnote entirely, because what follows is going to be exactly that.

This document quickly introduces the problem surrounding decimal numbers in calculators, discusses a way to harness more digits, a discussion of how it is implemented in C47/R47, and provides a number of detailed tutorial-style examples to work through to get used to the XFN special extended precision way of working.

## Table of Contents

|                                                                                      |           |
|--------------------------------------------------------------------------------------|-----------|
| <b>The C47/R47 1000-digit issue .....</b>                                            | <b>1</b>  |
| <b>Theoretical Decimal Numbers .....</b>                                             | <b>2</b>  |
| Decimal number background .....                                                      | 2         |
| Large-integer solution for trig-functions .....                                      | 2         |
| Extending to 1000-digit precision floating point.....                                | 3         |
| The treacherous last digit .....                                                     | 3         |
| Floating point in our calculators.....                                               | 3         |
| <b>Introducing extended precision (XFN) to C47/R47 .....</b>                         | <b>4</b>  |
| Composite data type for high precision numbers.....                                  | 4         |
| Example losing accuracy using a 10-digit HP calculator .....                         | 4         |
| Doubling registers to improve accuracy .....                                         | 4         |
| Storing the error of the first rounding, into the second register.....               | 5         |
| Example NOT losing accuracy using a 2 x 10-digit registers.....                      | 5         |
| <b>XFN functions in C47/R47 .....</b>                                                | <b>6</b>  |
| Example: practical exercise .....                                                    | 6         |
| <b>Composite FMA 1034-digit data type summary .....</b>                              | <b>7</b>  |
| <b>Examples .....</b>                                                                | <b>8</b>  |
| Obtain $\pi$ : 1000 digits the easy way with $[X\pi]$ .....                          | 8         |
| Obtain $\pi$ : 1000 digits with $XACOS(0)$ .....                                     | 8         |
| $\sin(\pi)$ to 1000 digits to verify the accuracy .....                              | 9         |
| The 9 calculator forensics of $\arcsin(\arccos(\arctan(\tan(\cos(\sin(9))))))$ ..... | 9         |
| Using standard trigonometry .....                                                    | 9         |
| Using XFN trigonometry: .....                                                        | 9         |
| Programmed example, of the calculator forensics test: .....                          | 10        |
| Practical examples using and rounding 1000 digits .....                              | 11        |
| Forum Example using XFN precision .....                                              | 11        |
| Numerical comparison using the $XCOS$ function in XFN: .....                         | 12        |
| Confirm that the WP43 integer range is based up to 39 integer digits: .....          | 13        |
| <b>XFN in USER mode using the labelled calculator keys:.....</b>                     | <b>14</b> |
| <b>Version history.....</b>                                                          | <b>15</b> |

# Theoretical Decimal Numbers

## Decimal number background

Floating-point numbers lose precision as soon as the digit limit is exceeded, whether by the user's input or by internal intermediate operations or functions.

Take, for example, a 4-digit calculator:  $9997 + 2 = 9.999 \times 10^3$ , but  $9999 + 2$  would be  $1.0001 \times 10^4$ . With only 4 digits available, the least significant digit is lost. The most significant digits are always kept, so  $9999 + 2$  is stored as  $1.000 \times 10^4$ , or 10000, not 10001.

Furthermore, any 4-digit number of the form  $n.nnn \times 10^m$  with  $m > 3$  is both an integer and a multiple of 10. For example,  $1.234 \times 10^4 = 12340$ , which is divisible by 10.

In a floating-point system, once the exponent exceeds the number of stored digits, integers can no longer be represented exactly. Integers that are exact multiples of powers of 10 are an exception, because the trailing zeros fall outside the stored digits. This allows a small set of very large integers with trailing zeros to be represented precisely.

## Large-integer solution for trig-functions

Integer inputs are defined in C47/R47 to be plus or minus 9.99...E+999. Although it does make little sense to find sin or cos of integer numbers of radians, users can and routinely do request that the calculator do examples for sine and cosine of large integers. Although finding solutions for large integer inputs to trigonometry was not originally included in the C47, integer input was later added to render correct 34-digit output for 1000-digit integer inputs.

The C47/R47 Long Integer input data type is suitable for this function, and the trigonometry functions automatically work and extend to the maximum range for any integer input.

Internally, we implemented  $2\pi$  modulus reduction with at least 2139-digit precision, enough to maintain accuracy for 1000-digit inputs. The implementation accepts integers in either 1000-digit Long Integer data type, or 34-digit floating-point Real type numbers with exponents between 33 and 999. Internal modulo  $2\pi$  reduction results in accurate trig function output of 34 digits.

**RAD 6381956970095103 ENTER 2 ENTER 797  $y^x \times \cos$**   
which evaluates precisely (rounded, 34-digits) to:  
**-4.687 165 924 254 627 611 122 582 801 963 884 E-19**

This method allows for accurate solutions of trig problems with extremely large integers, even when values lie very close to  $\pi$  boundaries, such as the example in [this thread](#) from @BigEd which was specifically chosen for entertainment value as the integer value is extremely close to an integer multiple of  $2\pi$ .

## Extending to 1000-digit precision floating point

Since the domain of large integers and integers divisible by 10 is vanishingly small, it makes more sense to extend precision to 1000-digit decimals rather than restrict it to integers.

### The treacherous last digit

Floating point numbers aren't new. Slide rules used a 2- or 3-digit floating point system with exponents in the user's imagination. That wasn't new either. Briggs<sup>1</sup> laid the groundwork in 1624 but before that Napier<sup>2</sup> coined logarithms in 1614. That was not new, Stifel and Bürgi separately started it even before. Yet it wasn't until 1972, that HP arrived to this party with floating point in a shirt pocket.

The advantages are obvious, but the problem is hidden: In your classic HP, consider an increment of the rightmost red LED 7-segment, or the *Unit in the Last Place* (ULP). This difference varies with the exponent and on the same machine it is 1E-9 for  $\pi$ , and 1E14 /mol for Avogadro's number. These are rather large differences that a single digit can make. ULP is literally the smallest representable change that one digit can make.

The key point is that the linear "number line" of the mantissa is uniform only for a fixed exponent and that tempts us to assume uniformity across all exponents, which it is not. Small numbers are tightly packed, while large numbers are spaced far apart and this warped line distorts arithmetic, causing issues like cancellation, loss of significance and more.

The only way we can improve the ULP to pack numbers tighter, is to increase the mantissa's digits. Expanding precision from 10 to 1000 digits costs 990 more digits in calculations, while the ULP improves from 1E-9 to 1E-999 for  $\pi$ , and from 1E14 to 1E-976 /mol for Avogadro's number, neither of which are practical use cases.

## Floating point in our calculators

Every calculator is limited by storage: a finite number of digits can be held or displayed.

In the C47/R47, we can see up to 20 digits out of the 34 stored internally. For most, arguably all, day-to-day problems, this is fine. If you set the display to fewer digits, say FIX 4 or SIG 5, the remaining hidden digits act as a safety net, protecting against the gradual digit corruption that can build up during long chains of operations.

To guard against digit corruption, we carry some extra digits internally which are not presented to the user. These digits shield the lower-order visible digits from rounding errors, keeping the 34-digit numbers more accurate. C47/R47 mostly works with 39 digits internally, but for specific functions more are required, such as 51, 75, 1071, or even 2139 digits used for *modulo*  $2\pi$  reduction.

Digit corruption would show up especially in chained operations, where successive calculations store intermediate results at 34-digit precision, not 39. Because the 34th digit is rounded at every step, the rounding error is cumulative, carried forward and can either progressively accumulate or partially cancel.

---

<sup>1</sup> Henry Briggs, [\*Arithmetica Logarithmica\*](#), 1624

<sup>2</sup> John Napier, [\*Mirifici Logarithmorum Canonis Descriptio\*](#), 1614

## Introducing extended precision (XFN) to C47/R47

The 1034-digit extended-accuracy XFN functions were developed for fun, requiring two technical aspects to be addressed: first, to raise the number of visible Real digits to 1000, and second, to add 34 guard digits in storage so that chained operations will better hold their accuracy.

In addition to the 1000 visible digits, 34 guard digits are stored and an additional 37 used to increase accuracy. In chained calculations, these guard digits protect the visible digits against precision loss and cumulative rounding errors.

XFN employs a composite register structure to achieve the 1034 stored digits detailed below.

### Composite data type for high precision numbers

In C47/R47, the composite data type comprises three stack registers to hold a 1034-digit structure using the Fused Multiplication and Addition ([FMA](#)<sup>34</sup>). The composite number comprises a 34-digit floating-point multiplier, a 1000-digit integer mantissa, and a 34-digit addition term (guard digits).

Storage is constrained to 1034 digits (1000-digit product plus 34-digit addition term).

#### *Example losing accuracy using a 10-digit HP calculator*

To illustrate the FMA concept using manageable 10-digit registers: consider a simple problem like:

$$((\pi \times 10^{-10})^2 \tag{1}$$

Suppose we use an HP-67 to calculate this: `SCI DSP 9  $\pi$  1 EEX 10 CHS  $\times$  ENTER  $\times$`  which would result in 9.869604404x10<sup>-20</sup> rather than the otherwise correctly rounded 9.869604401x10<sup>-20</sup>, with the digit 4 indicating incorrect rounding. This demonstrates lower order digit corruption due to the lack of guard digits.

Successive calculations allow small errors to accumulate (or cancel) and can gradually erode accuracy. The underlying problem is that  $\pi$  is not exactly 10 digits<sup>5</sup> 3.141592654. To improve accuracy, we would need more digits, say 20 instead of 10.

#### *Doubling registers to improve accuracy*

Doubling up on registers with an increase of  $\pi$  to 20 digits<sup>6</sup> as 2 groups of 10 digits per register, would round each register separately so the number cannot be preserved as a true 20-digit value.

---

<sup>3</sup> Nvidia, *Floating Point and IEEE 754 Compliance for NVIDIA GPUs*, [2.3 FMA](#)

<sup>4</sup> Wikipedia, [Fused multiply-add](#)

<sup>5</sup>  $\pi_{10}$  as determined by Madhava of Sangamagrama ca. 1400

<sup>6</sup>  $\pi_{20}$  as determined by Ludolph van Ceulen between 1596 and 1600

We cannot simply place the registers “head to tail,” because the midpoint being the 10th digit of the first register, will already have been rounded. The effect of double registers can be seen below where the red digit indicates rounding:

$$- \pi_{1-20} \text{ rounded} = 3.141\,592\,653\,589\,793\,238\,5 \quad (2)$$

$$- \pi_{1-10} \text{ rounded} = 3.141\,592\,65\mathbf{4} \quad (3)$$

$$- \pi_{11-20} \text{ rounded} = 589\,793\,238\,5 \quad (4)$$

### *Storing the error of the first rounding, into the second register*

Instead of splitting 20 digits into 2 x separate 10-digit groups, consider after creating and rounding the first Reg1, determine the rounding error and store that in Reg2.

$$- \text{Reg1} = \pi \text{ rounded to 10 digits} \quad (5)$$

$$- \text{Reg1} = \pi_{1-10} \text{ rounded} = 3.141\,592\,65\mathbf{4} \quad (6)$$

$$- \text{Reg2} = \delta = (\pi - \text{Reg1}) \text{ rounded to 10 digits} \quad (7)$$

$$- \text{Reg2} = \delta = -4.102\,067\,615\, \text{E-10} \, (\dots 615\,37) \quad (8)$$

-

$$- \text{Recompose value} = \text{Reg1} + \text{Reg2} = (\text{Reg1}) + (\pi_{20} - \text{Reg1}) \rightarrow \pi_{20} \quad (9)$$

$$- \pi_{20} \text{ reconstructed} = 3.141\,592\,65\mathbf{4} + (-4.102\,067\,615\, \text{E-10}) \quad (10)$$

$$- \pi_{20} \text{ reconstructed} = 3.141\,592\,653\,589\,793\,238\,5 \quad (11)$$

### *Example NOT losing accuracy using a 2 x 10-digit registers*

Considering problem (1), with two 10-digit registers Reg1 and Reg2 with automatic rounding applied to each. For example, for  $\pi$ , as above:

$$- \text{Reg1} = 1\text{E-10} \, \pi_{1-10} \text{ rounded} = 3.141\,592\,65\mathbf{4} \times 10^{-10} \quad (12)$$

Because of this rounding, Reg1 slightly exceeds  $\pi \times 10^{-10}$ , and is approximately 0.000 000 000 000 000 000 041 020 676 15... greater. If we would store the negative of this difference,

$$- \text{Reg2} = -4.102067615 \times 10^{-20}, \quad (13)$$

Reconstructing: [RCL] 1 [RCL] 2 [+] provides 1E-10  $\pi$  to 20 digits. The square function calculation rounded to 10 display digits would now yield 9.86960440 $\mathbf{1} \times 10^{-20}$ , restoring the correct last digit, due to the guard digits in Reg2 in place.

*This method scales up to the 1000-digit solutions below.*

## XFN functions in C47/R47

The following RPN functions are part of the [XFN for C47](#) and [XFN for R47](#) calculators:

### Entry:

X->XFN - Convert Integer/Real<sub>34</sub> in X to a XY+Z, Real<sub>1034</sub>

X $\pi$  - Constant  $\pi_{1034}$  to XY+Z, Real<sub>1034</sub>

### Trigonometry:

XSIN, XCOS, XTAN, XASIN, XACOS, XATAN, X=>DEG, X=>RAD, XDRG - Fn(XY+Z)  $\Leftrightarrow$  XY+Z  
Trig function from Real<sub>1034</sub>, to Real<sub>1034</sub>

XATAN2 - Fn(XY+Z, TA+B)  $\Leftrightarrow$  XY+Z  
Trig function from 2 $\times$ Real<sub>1034</sub>, to Real<sub>1034</sub>

### Arithmetic:

XCHS, XMOD<sub>4</sub> - Fn(XY+Z)  $\Leftrightarrow$  XY+Z  
Trig function from Real<sub>1034</sub>, to Real<sub>1034</sub>

XADD, XSUB, XMULT, XDIV, XPOWER, XXRTY, XMOD - Fn(XY+Z, TA+B)  $\Leftrightarrow$  XY+Z  
Trig function from 2 $\times$ Real<sub>1034</sub>, to Real<sub>1034</sub>

### Functions:

XSQR, XSQRT, XLOG, XLN, Xe^X, X10^X, X1/X - Fn(XY+Z)  $\Leftrightarrow$  XY+Z  
Trig function from Real<sub>1034</sub>, to Real<sub>1034</sub>

### Rounding:

XRDP n, XRSD n - Fn(XY+Z)  $\Leftrightarrow$  XY+Z  
Round from Real<sub>1034</sub>, to Real<sub>1034</sub>

[x][+] - (XY+Z) [x][+]  $\Leftrightarrow$  X  
Round from Real<sub>1034</sub>, to Real<sub>34</sub>

*Example: practical exercise*

Do the functions below in sequence and check the results:

*Tip: In the sim, to copy the mantissa in Y to clipboard: do X<>Y, Ctrl+c*

| Function | Result in XY+Z or X                                              | Real number scope     |
|----------|------------------------------------------------------------------|-----------------------|
| 2 X->XFN | XY+Z=2.0                                                         | 1 digit               |
| X $\pi$  | XY+Z=3.14159265358979323846264338327950288419716939937510582...  | 1000+34 digits        |
| XMULT    | XY+Z=6.28318530717958647692528676655900576839433879875021164...  | 1000+34 digits        |
| 1 X->XFN | XY+Z=1.0                                                         | 1 digit               |
| Xe^X     | XY+Z=2.71828182845904523536028747135266249775724709369995957...  | 1000+34 digits        |
| XADD     | XY+Z=9.00146713563863171228557423791166826615158589245017122...  | 1000+34 digits        |
| X=>RAD   | XY+Z=9.00146713563863171228557423791166826615158589245017122...r | 1000+34 digits        |
| XSIN     | XY+Z=0.41078129050290869547600949201836059188830697039341535...  | 1000+34 digits        |
| XRDP 40  | XY+Z=0.4107812905029086954760094920183605918883                  | Radix + 40 digits     |
| [x][+]   | X=0.4107812905029086954760094920183606                           | 34 digits             |
| c X->XFN | XY+Z=299 792 458.0                                               | 9 digits              |
| XSQRT    | XY+Z=17314.5158176600479837883397585479518804866489882885998...  | 1000+34 digits        |
| XRSD 04  | XY+Z=17310.0 = 1.731 $\times 10^{-5}$                            | 4 significant digits  |
| X1/X     | XY+Z=0.00005777007510109763142692085499711149624494511842865...  | 1000+34 digits        |
| XRDP 20  | XY+Z=0.00005777007510109763                                      | Radix + 20 digits     |
| XRSD 10  | XY+Z=0.00005777007510 = 5.777007510 $\times 10^{-5}$             | 10 significant digits |

## Composite FMA 1034-digit data type summary

The C47 on DM42n hardware and the R47 use the arbitrary precision *decNumber*<sup>7</sup> library and the calculator employs the special storage system described above, to process and store large decimals with 1000 visible digits using FMA. The composite number comprises (1) the mantissa, (2) the multiplication factor and (3) the difference term. All Log Integer or Real types.

See prior 10-digit example in (4) above: Enter the constituent parts of a composite number:

4.102067615 CHS EEX 20 CHS ENTER 3141592654 ENTER EEX 19 CHS ENTER

|            |                                          |      |      |    |
|------------|------------------------------------------|------|------|----|
| 2025-10-27 | °                                        | /64x | 64:2 | 0  |
| T×A+B=     |                                          |      |      | 3. |
| X×Y+Z=     | 0.000 000 000 314 159 265 358 979 323 85 |      |      |    |
|            | -4.102 067 615×10 <sup>-20</sup>         |      |      |    |
|            | Z: 3 141 592 654                         |      |      |    |
|            | 1.×10 <sup>-19</sup>                     |      |      |    |

$\pi \times 10^{-10}$  to 20 digits using 10-digit registers:  
Three C47/R47 storage registers to form a large decimal using the FMA concept:

$$[1 \times 10^{-19}] \times [3\ 141\ 592\ 654] + [-4.102\ 067\ 615 \times 10^{-20}] = 3.141\ 592\ 653\ 589\ 793\ 238\ 5 \times 10^{-10}$$

Example (4): Extended precision  $\pi_{20}$

X×Y+Z is used in C47/R47 to store a number up to 1034-digits, displaying 1000 digits with a built-in 34-digit addition register used for 34 stored guard digits in chain operations!

For the 'square problem' in (1) above, to use the C47/R47 to do  $(\pi \times 10^{-10})^2$ , instead of the HP-67, use the following keystrokes:

|                                                                                                                    |
|--------------------------------------------------------------------------------------------------------------------|
| X.FN [XFCNS] [X $\pi$ ] -10 [10 <sup>x</sup> ] 0 ENTER ENTER<br>[XMULT]<br>2 ENTER 0 ENTER ENTER [↑] [XPOWER] SHOW |
| X.FN [XFCNS] [X $\pi$ ] -10 [10 <sup>x</sup> ] [X→XFN]<br>[XMULT]<br>2 [X→XFN] [XPOWER] SHOW                       |

|            |                                                                     |      |      |   |
|------------|---------------------------------------------------------------------|------|------|---|
| 2025-10-27 | °                                                                   | /64x | 64:2 | S |
| X: XY+Z =  | 9.869 604 401 089 358 618 834 490 999 876 151 135 313 699           |      |      |   |
|            | 407 240 790 626 413 349 376 220 044 822 419 205 243 001 773 403 718 |      |      |   |
|            | 552 231 824 025 913 774 023 144 077 772 348 122 030 046 727 610 617 |      |      |   |
|            | 677 985 197 660 990 399 856 206 575 630 571 506 041 232 840 328 780 |      |      |   |
|            | 869 352 769 342 164 939 666 571 519 044 538 735 261 779 413 820 258 |      |      |   |
|            | 260 581 693 412 515 592 048 309 818 873 270 033 076 266 671 104 358 |      |      |   |
|            | 950 871 504 100 325 788 536 595 276 357 752 837 922 683 318 745 086 |      |      |   |
|            | 404 546 354 125 026 973 729 566 958 334 227 858 150 006 365 227 095 |      |      |   |
|            | 472 490 859 756 072 669 264 752 779 005 285 336 452 206 669 808 264 |      |      |   |
|            | 158 968 771 057 327 889 291 746 901 545 510 069 254 432 457 036 449 |      |      |   |
|            | 656 172 537 928 607 606 008 145 972 589 229 232 414 240 044 295 981 |      |      |   |
|            | 361 814 413 706 777 778 194 739 658 303 170 856 632 789 570 753 407 |      |      |   |
|            | 991 714 523 158 926 372 114 463 828 264 432 852 803 792 850 348 095 |      |      |   |
|            | 233 899 503 968 574 609 485 346 009 017 742 932 205 799 035 917 357 |      |      |   |
|            | 820 465 758 041 931 686 823 002 196 146 899 270 420 614 296 963 466 |      |      |   |
|            | 005 799 840 351 642 136 543 049 984 533 721 736 557 240 463 676 848 |      |      |   |
|            | 876 261 512 299 027 059 938 010 299 446 886 181 716 260 980 130 876 |      |      |   |
|            | 530 037 060 158 369 198 676 286 005 079 936 468 322 669 731 568 367 |      |      |   |
|            | 175 558 971 198 752 975 296 394 916 315 539 449 195 483 877 068 721 |      |      |   |
|            | 130 789 866 575 590 986 536 365 630 763 630 880 612×10 <sup>0</sup> |      |      |   |

$(\pi \times 10^{-10})^2$  as 1000-digit composite using SHOW

|                   |            |                                               |      |      |   |
|-------------------|------------|-----------------------------------------------|------|------|---|
| Composite numbers | 2025-10-27 | °                                             | /64x | 64:2 | S |
| Z Reg. 34d        | T×A+B=     | 3.141 592 653 589 793 238 462 643 383 279 503 |      |      |   |
| Y Reg. 1000d      | X×Y+Z=     | 9.869 604 401 089 358 618 834 490 999 876 151 |      |      |   |
| X Reg. 34d        |            | -4.991 646 260 817 39×10 <sup>-1000</sup>     |      |      |   |
|                   | Z:         | 9.869 604 401 089 36×10 <sup>999</sup>        |      |      |   |
|                   |            | 1.×10 <sup>-999</sup>                         |      |      |   |

Viewer:  $\pi$  &  $(\pi \times 10^{-10})^2$ : rounded 34-digit composites

Stack:  $(\pi \times 10^{-10})^2$ : 1034-digit constituent registers

<sup>7</sup> Mike Cowlishaw, [The decNumber C library, Version 3.68, 2010](#)



## sin(π) to 1000 digits to verify the accuracy

[RAD] set to radians ADM  
 [Xπ] get the π constant into XYZ  
 [XSIN] obtain the sine of XYZ (composite value π)  
 Result is  $0.0 + 1.817... \times 10^{36} \times -1. \times 10^{-1070}$   
 [×][+]  $-1.817... \times 10^{-1034}$  as expected, almost 0

```

2025-10-28  L  ° /64×  64:2  0.
T×A+B=  3.141 592 653 589 793 238 462 643 383 279 503
X×Y+Z= -1.817 203 176 969 804 796 469 814 703×10-1034
Z: 1.817 203 176 969 805×1036
      -1.×10-1070
  
```

This composite result means that the result is 0.0... for 1033 digits, then 181 720 317 696 980 5..., so in short, the result is:

0., 1033 0's, and the 1034<sup>th</sup> digit is 1!

## The 9 calculator forensics<sup>8</sup> of arcsin(arccos(arctan(tan(cos(sin(9))))))

This test is a precision **torture test** that chains six trigonometric functions and their inverses starting from 9 to expose accumulated rounding errors and algorithmic quality by measuring how close the result gets back to 9.

### Using standard trigonometry

[DEG] 9 ENTER [SIN] [COS] [TAN] [ATAN] [ACOS] [ASIN]

The result is 9, and with SHOW, exposing all 34 digits: 8.999 999 999 999 999 999 999 999 999 999 999 937 535°, thus accurate to 28 digits to 9.0.

```

2025-09-20  L  ° /64×  64:2  8.
X:8.999 999 999 999 999 999
      999 999 999 937 535 °
X: 8°59' 59.99"
X:1.570 796 326 795 0×10-1 r
      X:0.05 πr
      X:10. 9
  
```

Result SHOW: 9° - 6.247 × 10<sup>-29</sup>

The reason for the inaccuracy of -6.247E-29 above, is that C47/R47 stores each interim result after each of the 6 trigonometric functions as a 34-digit real number where the 34th digit after each calculation is rounded. Every successive function therefore receives a rounded number, performs the function and rounds the output by max. on more rounded number. This result has 5-rounded values in this case.

The XFN method improves on this by storing not only the guaranteed 1000 digits interim values, but also 34 guard digit remainders, essentially storing accurate buffer digits so that every 1000-digit trig operation is stored accurate due to the 34 buffer digits.

### Using XFN trigonometry:

Operations using extended accuracy: [DEG] 9 [X→XFN] [XSIN] [XCOS] [XTAN] [XATAN] [XACOS] [XASIN]

The result is 9, accurate to 1027 digits. It is displayed to 1000 digits using SHOW 9, rounded to 9° - 1.3×10<sup>-1028</sup>.

```

2025-10-28  L  ° /64×  64:2  0.
T×A+B=  9.
X×Y+Z= -1.310 8×10-1028
      9
      1. °
  
```

Result 9 × 1° - 1.3 × 10<sup>-1028</sup>.

<sup>8</sup> Mike Sebastian's calculator forensics test, by entering 9 through sin, cos, tan, and their inverses, reveals subtle floating-point errors and algorithmic differences considering deviations from 9. ([Table](#), [Web site](#))

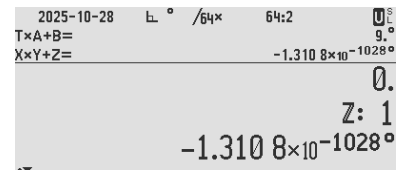
*Programmed example, of the calculator forensics test:*

The programmed operation below illustrates extended accuracy (XFN), with 9 is subtracted from the result 9: on the right:

```

0001: LBL 'FORENSI'
0002: DEG
0003: 9
0004: X→XFN      Convert 9 in X to a triple register
0005: 3STO G      STO X, Y & Z to triple register G, H & I
0006: XSIN        Extended accuracy sine
0007: XCOS        Extended accuracy cosine
0008: XTAN        Extended accuracy tangent
0009: XATAN       Extended accuracy arc tangent
0010: XACOS       Extended accuracy arc cosine
0011: XASIN       Extended accuracy arc sine
0011: 3RCL G      Recall triple register stored in G, H & I
0012: XSUB        Extended accuracy subtract
0013: END

```



Programmed example, 9 subtracted from the result; the remainder is the same as Z of course:  $-1.3 \times 10^{-1028}$

## Practical examples using and rounding 1000 digits

### Forum Example using XFN precision

An extended accuracy problem was discussed in a [forum thread](#), to mimic the WP43 v0.29 calculation mathematically, where it is known that this version of WP43 loses precision for large integer inputs to the cosine function when the integers exceed the 34-digit floating point decimal range.

In the [forum thread](#) above where the old version 0.29 was discussed, I solved the mimic operation using WolframAlpha but in this appnote exercise I solve the same using the C47/R47 XFN extended accuracy. The stated problem was to solve  $\cos(638\ 195\ 697\ 009\ 510\ 3 \times 2^{797})$ , confirm and explain the results obtained on WP43 by mimicking the math.

The first step is to confirm the standard 34-digit cos-output for the 256-digit integer input:

|                                              |                                                                                                                                                                                                                                                                                  |   |
|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| WP43                                         | 638 195 697 009 510 3 ENTER<br>2 ENTER<br>797 EXP [y^x] [*]                                                                                                                                                                                                                      | 1 |
| Result in<br>X: 256-digit<br>Long<br>Integer | X = 5319372648326541416707296656673541083813475031793921822105998164685326<br>343987747477646239125204069843392466931105720371047561653378447496736288905<br>533500277726150903890962697774418679535123008556835980236851047840822029788<br>166318932319835828816270258618761216 | 2 |
|                                              | [MODE] [RAD] TRI [cos]                                                                                                                                                                                                                                                           | 3 |
| Std 34-<br>digit                             | X = -0.999 440 844 081 712 927 910 734 972 739 862                                                                                                                                                                                                                               | 4 |

It was confirmed in multiple prior posts that WP43 v0.29 does not accurately calculate ridiculously large inputs to cos and other trigonometric functions. Result (4) is due to the 256-digit input angle rounded to 39 significant digits prior to the modulo  $2\pi$  reduction. Compare the discrepancy of WP43 (4) with (17) being the accurate result.

For later correlation, we also determined the actual internal WP43 input range reduction:

|                 |                                                                                                                                               |   |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------|---|
| Using<br>debug: | Internal reduced angle handed from the angle range reduction:<br>0.033 442 728 216 515 956 875 699 096 225 491 439 634 906 964 697 14 radians | 5 |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------|---|

To mimic the WP43 numerical process, we follow the process to convert the 256-digit integer to a 39-digit *decNumber*<sup>9</sup> Real. The first 39 digits of the Long Integer are retained in the Real type, with the rest (205 digits) defaulted to zero:

|                     |                                                                                                                                                         |   |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| C47 XFN:            | 638 195 697 009 510 3 ENTER 2 ENTER 797 [y^x] [*] [X→XFN] [XRSD] 39                                                                                     | 8 |
|                     | Create the Long Integer input in X, convert it to a composite in XY+Z.<br>Round the result to 39 digits, losing all digits after the 39 <sup>th</sup> . |   |
| 39-digit<br>rounded | 5.31937264832654141670729665667354108381 × 10 <sup>255</sup>                                                                                            | 9 |

The 39-digit integer Real number (9) would internally in the WP43, be range reduced to the 0 -  $2\pi$  range using modulo  $2\pi$ , then to the 0 -  $\pi/2$  range using trigonometric identities:

<sup>9</sup> Mike Cowlshaw, [The decNumber C library. Version 3.68, 2010](#)

|                                 |                                                                                                                                                            |     |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
|                                 | [XMOD <sub>4</sub> ] [XRSD] 51                                                                                                                             | 10  |
| Reduce                          | Do an accurate 2139+ digit modulo $2\pi$ reduction, and round the result to 51 significant digits:                                                         |     |
| 51-digit rounded in $[0-2\pi)$  | 3.17503538180630919533834247950499432383207636407225                                                                                                       | 11  |
|                                 | [X $\pi$ ] [XSUB] [XRSD] 51                                                                                                                                | 12  |
|                                 | Reduce the result to within $[0-\pi/2)$ using a quadrant reduction identity by subtracting $\pi$ from the result and again round to 51 significant digits: |     |
| 51-digit rounded in $[0-\pi/2)$ | The 51-digit number, range reduced to $0-\pi/2 =$<br>0.033 442 728 216 515 956 875 699 096 225 491 439 634 906 964 697 144 2 ✓                             | 13  |
|                                 | Compare with (5) captured from WP43:<br>0.033 442 728 216 515 956 875 699 096 225 491 439 634 906 964 697 14 ✓                                             | (5) |

The WP43 internal reduced number agrees with our mimicked reduced angle.

To complete the mimic of what WP43 would produce, we need to draw a cosine on the computed 51-digit reduced angle and round that to a 34-digit number:

|                     |                                                                                                      |     |
|---------------------|------------------------------------------------------------------------------------------------------|-----|
| Convert to 34-digit | [XCOS] [XRSD] 34 [*] [+] [CHS]                                                                       | 14  |
|                     | From (13) do a cosine, bearing in mind the sign change required for the quadrant reduction identity: |     |
| 34-digit rounded    | X = -0.999 440 844 081 712 927 910 734 972 739 862 ✓                                                 | 15  |
| 34-digit            | COMPARE WITH (4) FROM WP43:<br>X = -0.999 440 844 081 712 927 910 734 972 739 862 ✓                  | (4) |

- *Verdict:*  
The WP43 v0.29 cosine for large integers is fully explainable as mimicked above.

#### Numerical comparison using the XCOS function in XFN:

For comparison from the XCOS function, the answer accurate to 1000 digits, rounded to 34 digits is:

|                               |                                                                                                                                                                                                                                      |    |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| C47 XFN:                      | 638 195 697 009 510 3 ENTER 2 ENTER 797 [y^x] [*] [X→XFN] [RAD] [XCOS] [*] [+]                                                                                                                                                       | 16 |
|                               | Create the Long Integer input in X, convert it to a composite in XY+Z. Do the extended precision cosine on it and convert the FMA number to a simple 34-digit Real number.                                                           |    |
| 34-digit rounded              | X = -4.687 165 924 254 627 611 122 582 801 963 884E-19 ✓                                                                                                                                                                             | 17 |
| C47:                          | 638 195 697 009 510 3 ENTER 2 ENTER 797 [y^x] [*] [RAD] [COS]                                                                                                                                                                        | 18 |
| 34-digit                      | X = -4.687 165 924 254 627 611 122 582 801 963 884E-19 ✓                                                                                                                                                                             | 19 |
| <a href="#">Wolfram Alpha</a> | Cos @ N[Mod[6381956970095103*2^797, Floor[N[2 Pi, 2139], 10^-2138]], 54], WorkingPrecision<br>(Adjusted the mod output to 54 digits to result in 34 digit result)<br>(WorkingPrecision is a hack to evaluate and needs no parameter) | 20 |
| WA                            | -4.687 165 924 254 627 611 122 582 801 963 884×10^-19 ✓                                                                                                                                                                              | 21 |

Confirm that the WP43 integer range is based up to 39 integer digits:

To prove my theory in the discussion after (4) above, I set out below to prove that Long Integers up to 39 digits would have accurate cosine results, but once exceeding 39 digits, the cosine should fail due to the rounding prior to internal range reduction:

To really stress test the modulo reduction, an integer that is extremely close to a multiple of  $2\pi$  is needed, and I searched for and found integer numbers  $N$ , which exceed 34 digits, do not exceed 39 digits, and for which also the following condition is true:

$| \text{mod}(N, 2\pi) - \pi |$  is very small:

1. 35 digits: 999 999 999 999 999 968 633 661 079 180 873 32
2. 39 digits: 9E38 + 404084116
3. 40 digits: 1E39 + 394950963

1. Try a 35-digit number, which works ✓

|          |                                                                               |    |
|----------|-------------------------------------------------------------------------------|----|
| C47 XFN: | 999 999 999 999 999 968 633 661 079 180 873 32 [X→XFN] [RAD] [XCOS] [XRSD] 50 | 22 |
|          | XY = -0.999 999 999 999 963 911 393 057 383 617 706 279 396 776 994 850 99 ✓  | 23 |
| WP43     | 999 999 999 999 999 968 633 661 079 180 873 32 [MODE] [RAD] TRI [cos]         | 24 |
|          | X = -0.999 999 999 999 963 911 393 057 383 617 706 3 ✓                        | 25 |

2. A 39-digit number works ✓

|          |                                                                              |    |
|----------|------------------------------------------------------------------------------|----|
| C47 XFN: | 38 [10^x] 9 [x] 404 084 116 [+] [X→XFN] [RAD] [XCOS] [XRSD] 50               | 26 |
|          | XY = -0.999 999 999 999 999 999 901 881 313 205 654 850 245 305 803 729 09 ✓ | 27 |
| WP43     | 38 [10^x] 9 [x] 404 084 116 [+] [MODE] [RAD] TRI [cos]                       | 28 |
|          | X = -0.999 999 999 999 999 999 901 881 313 205 654 9 ✓                       | 29 |

3. A 40-digit number does not work ✗

|          |                                                                              |    |
|----------|------------------------------------------------------------------------------|----|
| C47 XFN: | 39 [10^x] 394 950 963 [+] [X→XFN] [RAD] [XCOS] [XRSD] 50                     | 30 |
|          | XY = -0.999 999 999 999 999 986 934 445 338 684 711 191 236 409 343 456 12 ✓ | 31 |
| WP43     | 39 [10^x] 394 950 963 [+] [MODE] [RAD] TRI [cos]                             | 32 |
|          | X = 0.989 992 495 879 059 765 996 702 351 529 824 1 ✗                        | 33 |

- Verdict:  
The WP43 V0.29 trigonometric functions (with internal modulo  $2\pi$  reduction), are within the accuracy provided for integers not exceeding 39 digits.

## XFN in USER mode using the labelled calculator keys:

A user key-only state file is included in the distribution, located in the STATE directory. The files **xfnToC47user.s47** can be loaded for the C47 and **xfnToR47user.s47** for the R47. These files load all user keys with the layouts shown below. Loading them will overwrite all existing user key allocations, so save your current state if you want to keep it.

The supplied state files only affect user keys and will not modify programs or calculator settings. To load one, use **g[I/O] → LOADST** and select the appropriate file. Once loaded, switch the calculator to **USER** mode to access the new key layouts.

You can easily enter a single real or integer value into a triple register using **[X→XFN]**. Use **3STO** or **3RCL** to store or recall an entire triple register set. For example, entering **3STO 50** saves the X, Y, and Z values to registers 50, 51, and 52 respectively.



Usage: Calculate  $3^{-4}$  in full precision: 3 [X→XFN] 4 [X→XFN] [XCHS] f[XPOWER] f[SHOW]

C47: press the following buttons in USER mode: 3 [Σ+] 4 [Σ+] [CHS] f[y<sup>x</sup>] f[SHOW]

R47: press the following buttons in USER mode: 3 g[R↑] 4 g[R↑] [CHS] [y<sup>x</sup>] f[SHOW]

**The C47 / R47 reference pages with advice on all the available commands can be found on the documentation section of the web site [47calc.com](http://47calc.com), specifically for [C47](#) and [R47](#).**

## Version history

Appnote 16                      Version 0            2025-10-28

Appnote 16a

Version 1            2026-02-01 Major rework of full document, including changes for R47

Version 2            2026-02-06 Correction ULP in “The treacherous last digit”, fixed spelling, fixed formatting.

Authors: Jaco Mostert, Robbert Jan van Meenen; Copyright (C) 2026 The C47 Development Team. Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license can be downloaded from [gnu.org](http://gnu.org).